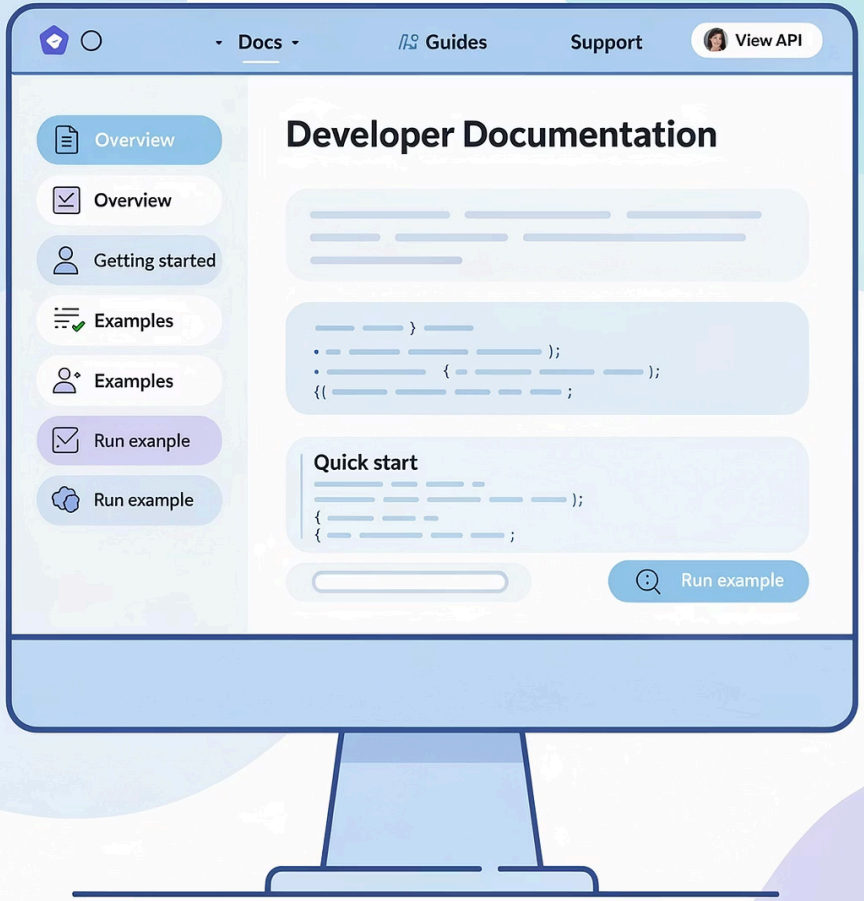
PHPSage

Josemi Rodríguez · Desarrollador Drupal · LinkedIn

Apasionado por la tecnología y movido por una curiosidad constante que me impulsa a aprender cada día.

Me entusiasma especialmente todo lo relacionado con, **Drupal**, **cloud**, **automatización** y **cultura DevOps**: diseñar flujos, simplificar operaciones y conectar el desarrollo con el despliegue de forma eficiente y casi mágica.

Actualmente trabajo como desarrollador Drupal en **idealista**.



Documentación completa del proyecto

PHPSage cuenta con **documentación pública y centralizada en GitHub Pages**, accesible para cualquiera que quiera explorar el proyecto en profundidad.

https://thebrokenbrain.github.io/phpsage/

Visión general

Contexto y motivación del proyecto

Arquitectura

Diseño técnico y decisiones clave

UX y API

Interfaz de usuario y endpoints

Despliegue

Infraestructura y demo pública

PHPSage en una frase

> PHPSage es una plataforma web que toma la salida de PHPStan y la transforma en una experiencia más **navegable, operativa y enriquecida con IA**.

No se limita a mostrar errores. PHPSage convierte el análisis estático en un flujo de trabajo completo que va más allá del listado de problemas.

Organiza resultados

Estructura los issues de forma navegable y comprensible

Navega por archivos

Explora directamente el código fuente afectado

Explica issues

La IA traduce los errores a lenguaje comprensible

Propone correcciones

Sugiere fixes razonados y contextualizados

El problema que intenta resolver

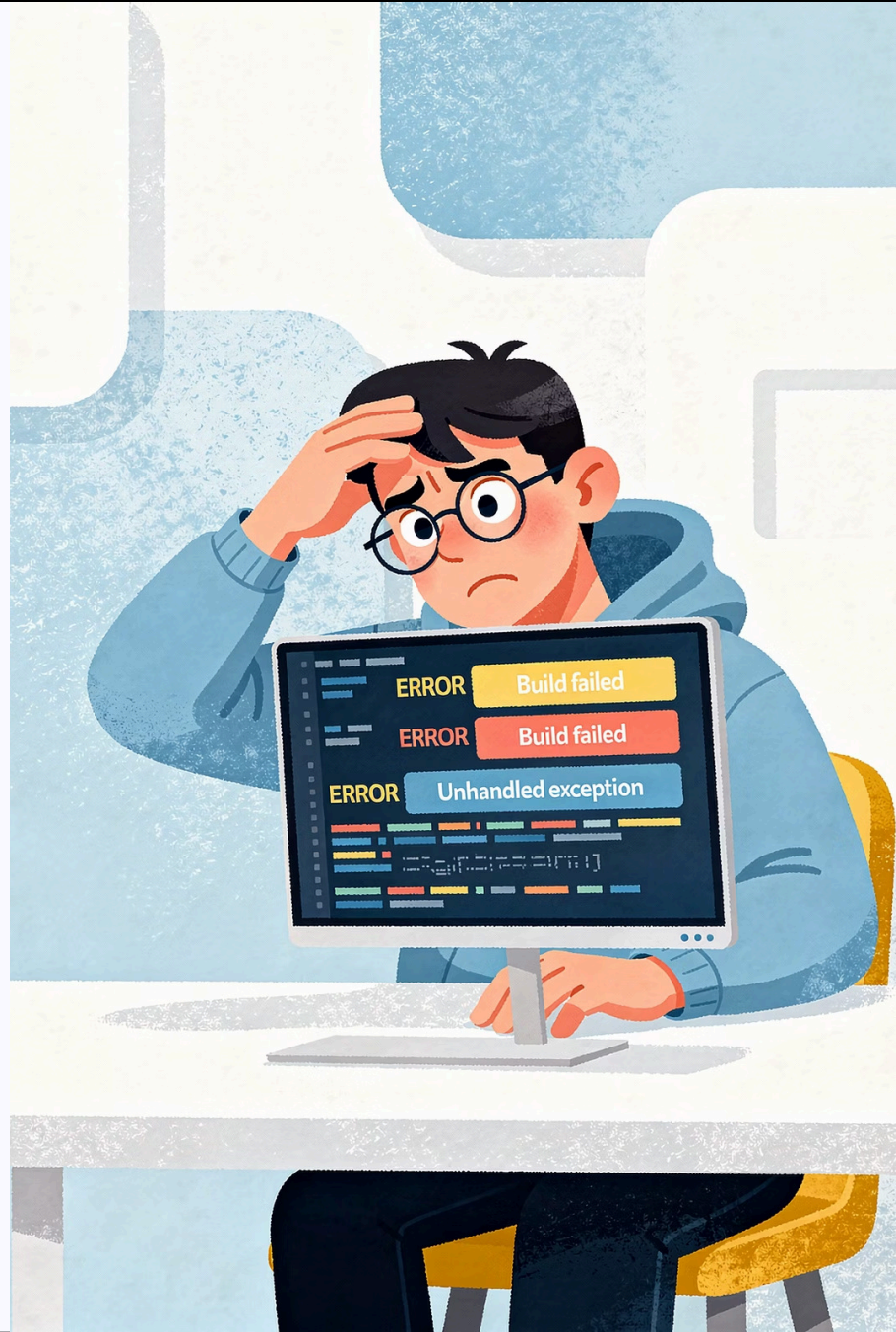
PHPStan es una herramienta muy potente para el análisis estático de PHP. Sin embargo, su salida tiene limitaciones importantes cuando se trabaja con proyectos reales a gran escala.

La salida de PHPStan

- Técnica y plana
- Poco navegable
- Difícil de escalar con muchos errores
- Sin jerarquía visual ni priorización

Fricción en el día a día

- Cuesta entender algunos mensajes de error
- Cuesta priorizar qué corregir primero
- Cuesta moverse entre archivos afectados
- No existe orientación clara para corregir cada issue



La propuesta de valor

PHPSage no reemplaza a PHPStan: lo **complementa y amplifica**. Añade tres capas sobre el análisis estático para convertirlo en una experiencia verdaderamente útil para desarrolladores, revisores técnicos y entornos docentes.

Navegación estructurada

Resultados organizados, filtrables y explorables desde una interfaz web clara

Asistencia con IA

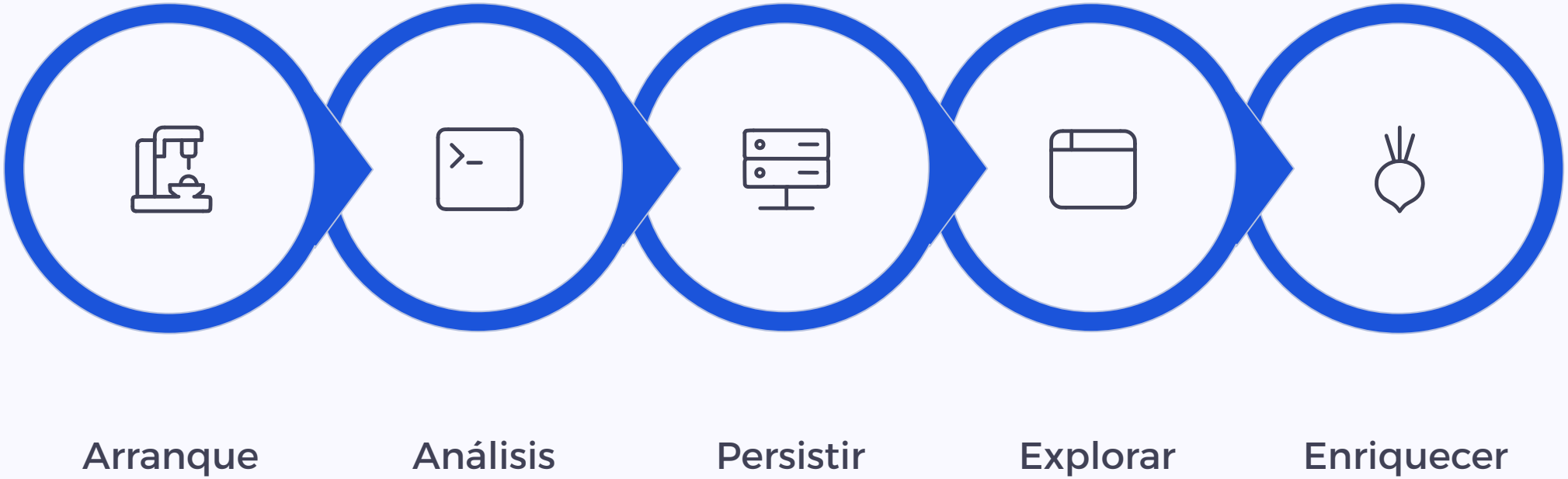
Explicaciones automáticas de cada issue y sugerencias de corrección precisas

Contexto documental con RAG

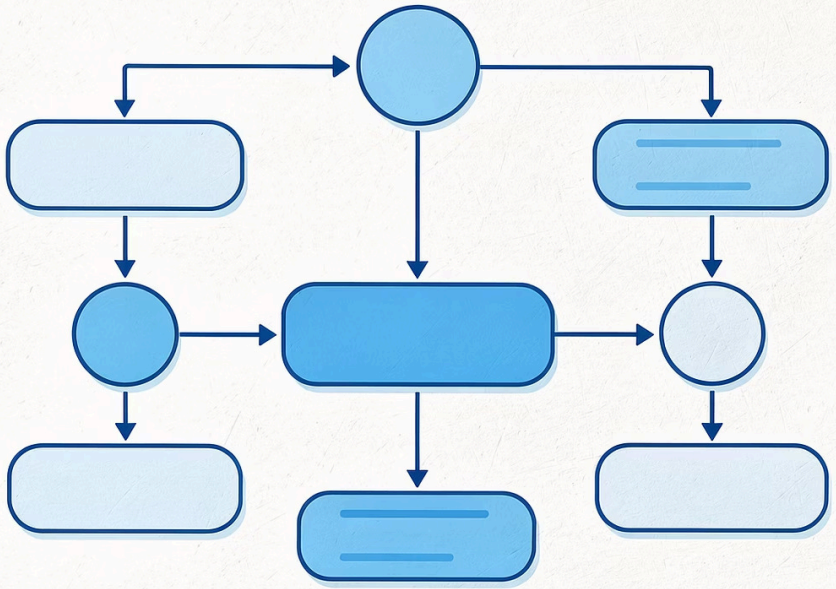
Respuestas enriquecidas con documentación específica de PHPStan para mayor precisión

Flujo principal del sistema

El ciclo de vida completo de un análisis con PHPSage sigue un flujo sencillo pero potente: desde el arranque de la plataforma hasta la exploración enriquecida de cada issue.



PHPStan genera el análisis y PHPSage lo convierte en una **experiencia navegable y explicable**. Cada pieza del sistema tiene una responsabilidad clara dentro del flujo.



Componentes principales

PHPSage está organizado como un **monorepo** con varias piezas que colaboran entre sí, y se integra con servicios externos según el modo de uso configurado.

Piezas internas del monorepo

CLI

Punto de entrada para lanzar análisis e ingestas

Server API

Gestión del ciclo de vida de los runs

Web UI

Interfaz de exploración de resultados

Servicios externos integrados

PHPStan

Motor de análisis estático

OpenAI

IA en la nube

Ollama

IA local sin dependencias externas

Qdrant

Base de datos vectorial para RAG

Stack tecnológico

PHPSage combina múltiples capas tecnológicas modernas para cubrir desde la lógica de aplicación hasta la infraestructura cloud, pasando por las capacidades de IA.

⚙️ Aplicación

- TypeScript
- Node.js
- React + Vite
- npm workspaces

🤖 IA y RAG

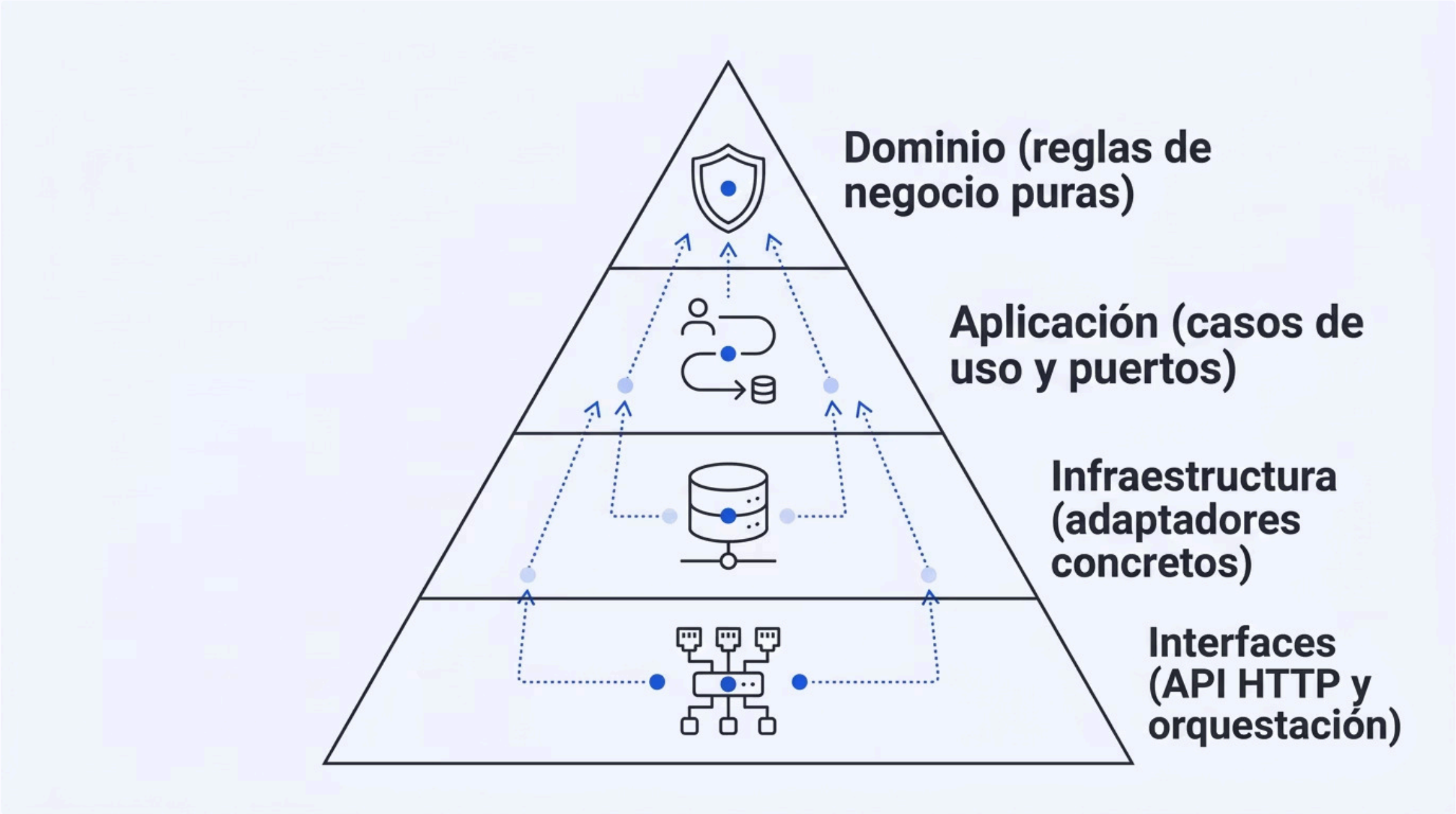
- OpenAI
- Ollama
- Qdrant
- Filesystem como backend simple

☁️ Infraestructura

- Docker / Compose
- Pulumi
- Hetzner Cloud
- Cloudflare + Traefik

Decisión arquitectónica

El servidor sigue una aproximación de **Clean Architecture**. La regla central es que las dependencias siempre apuntan hacia dentro, lo que mantiene el sistema claro, testeable y preparado para evolucionar.



Esta estructura permite cambiar adaptadores de infraestructura –base de datos, proveedor de IA, almacenamiento– sin tocar la lógica de negocio. Una inversión de complejidad que paga dividendos a largo plazo.

Cómo se usa PHPSage

PHPSage expone tres puntos de entrada complementarios para cubrir distintos momentos del flujo de trabajo: automatización, integración y exploración visual.

Puntos de entrada

- **CLI** – lanzar análisis e ingestas desde la terminal
- **API HTTP** – gestionar el ciclo de vida de los runs programáticamente
- **UI Web** – explorar resultados de forma visual e interactiva

Comandos principales

```
phpsage app
phpsage phpstan analyse <path>
phpsage rag ingest
```

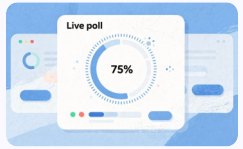
Con estos tres comandos es posible levantar la plataforma, ejecutar un análisis completo e ingestar documentación para el sistema RAG.

Experiencia de usuario: Dashboard

El Dashboard es la **vista operativa principal** de PHPSage. Desde aquí el desarrollador tiene control total sobre el estado de los análisis y acceso directo a toda la información relevante.

Gestión de runs

Ver lista de runs, lanzar nuevos análisis, consultar logs, revisar issues y explorar archivos afectados con preview del código fuente.



Funcionalidades operativas

Live polling, estado de IA en tiempo real y panel de jobs de ingesta para un control completo del ciclo de vida.

Experiencia de usuario: Insights

La vista Insights está orientada a la **lectura rápida y el análisis agregado**. Permite entender de un vistazo la distribución de errores y su impacto sobre el proyecto analizado.

Métricas principales

$\sum$

Total issues

Número total de problemas detectados en el análisis

#1

Más frecuente

Identificador con mayor número de ocurrencias

Qué aporta esta vista

- Identificadores únicos detectados
- Cobertura de identificadores conocidos
- Visualizaciones de distribución de errores
- Enlace a documentación de PHPStan cuando existe contexto conocido

Una capa de agregación que transforma datos brutos en **inteligencia accionable**.

Experiencia de usuario: Issue

La vista Issue **baja al máximo detalle**. No se limita a mostrar dónde está el error: lleva al desarrollador directamente al código, con contexto, y le ofrece asistencia inteligente para entenderlo y corregirlo.

Localización exacta

Archivo, línea afectada y navegación directa al código con la línea resaltada y contexto visible

Explain

La IA explica en lenguaje claro qué significa el issue y por qué se produce

Suggest Fix

Propone una corrección razonada y alineada con el contexto real del código

La intención es que el desarrollador no solo *vea* el error, sino que lo **entienda y sepa cómo resolverlo**.

IA y RAG

La capa de IA de PHPSage no funciona de forma aislada: se apoya en **contexto documental específico** para ofrecer respuestas más precisas y útiles que las de un modelo genérico.

Sistema RAG basado en documentación de PHPStan

PHPSage ingesta documentación específica de errores de PHPStan en una base de datos vectorial (Qdrant), lo que permite recuperar contexto relevante antes de generar una respuesta.

Respuestas más precisas y razonadas

Menos explicaciones genéricas, fixes más fundamentados y mejor alineación con el contexto real del issue analizado.

Principio de conservadurismo

El sistema es deliberadamente cauteloso con los parches: **es mejor no proponer un fix que mostrar uno inseguro**. La confianza del desarrollador es prioritaria.

Demo pública y despliegue real

PHPSage no se queda solo en local. El proyecto dispone de una **demo desplegada en remoto**, accesible por HTTPS y protegida con Cloudflare Zero Trust. Esto demuestra que no es solo una prueba de concepto.

Hetzner Cloud

Infraestructura cloud sobre la que corre la plataforma desplegada

Cloudflare

DNS, acceso seguro y protección con Zero Trust

Pulumi + Docker Compose

Automatización de infraestructura y operación del sistema

Qué se puede validar en la demo

La demo pública es el punto más directo para revisar el **comportamiento real del proyecto** sobre un análisis auténtico con el proyecto de ejemplo `examples/php-sample`.

1. **Plataforma desplegada y accesible**
   La UI está operativa sobre infraestructura real, no sobre un entorno simulado

2. **Análisis real cargado**
   Los resultados provienen de un análisis PHPStan real sobre código PHP de ejemplo

3. **Integración con OpenAI activa**
   La asistencia IA funciona en remoto, con Explain y Suggest Fix operativos

4. **Experiencia completa fuera de local**
   Todo el flujo –análisis, navegación e IA– funciona de extremo a extremo en producción

Cierre

> PHPSage nace de una idea sencilla: hacer que el análisis estático de PHP sea más **comprensible, navegable y accionable**.

Técnica

Arquitectura limpia, TypeScript, React, Docker y despliegue cloud real

Experiencia

UX cuidada, flujos operativos y asistencia IA integrada

Personal

La intersección entre desarrollo, DevOps, cloud y las ganas de construir herramientas útiles de verdad

Este proyecto representa, también, algo personal: una demostración de cómo **desarrollo, curiosidad técnica y cultura DevOps** pueden confluir en una herramienta real, desplegada y completa.